

Reinforcement Learning

CS 59300: RL1

October 23, 2025

Joseph Campbell
Department of Computer Science

Project proposals due Oct 30

2-page research-style paper outlining your project and progress

IEEE-style format

Template:

<https://ras.papercept.net/conferences/support/support.php>

Midterm November 4

Will release “high-level” study guide
In class at regular time and location

Topics:

- Bandits/MDPs
- Bellman equations, policy/value iteration
- DQNs
- Actor-Critic
- Multi-Agent RL
- Exploration

Today's lecture

1. Exploration continued
2. Inverse reinforcement learning
2. Generative adversarial imitation learning

Some content inspired by Sergey Levine's Berkeley CS285

Exploration continued

Curiosity

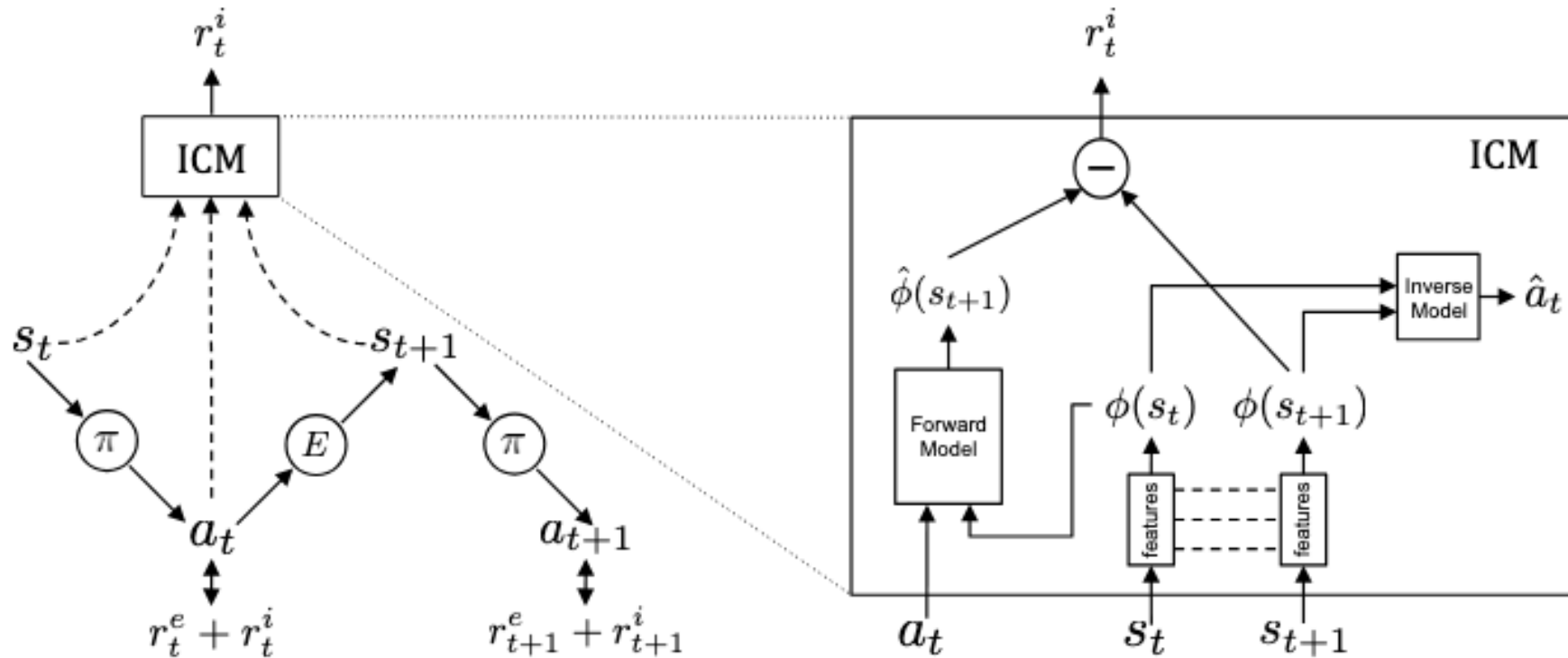
The agent learns to predict the outcome of its actions

- Visual forward dynamics
- Self-supervised learning (no labels!)

Agent takes an action and compares its predicted outcome to actual

- The error is the exploration bonus

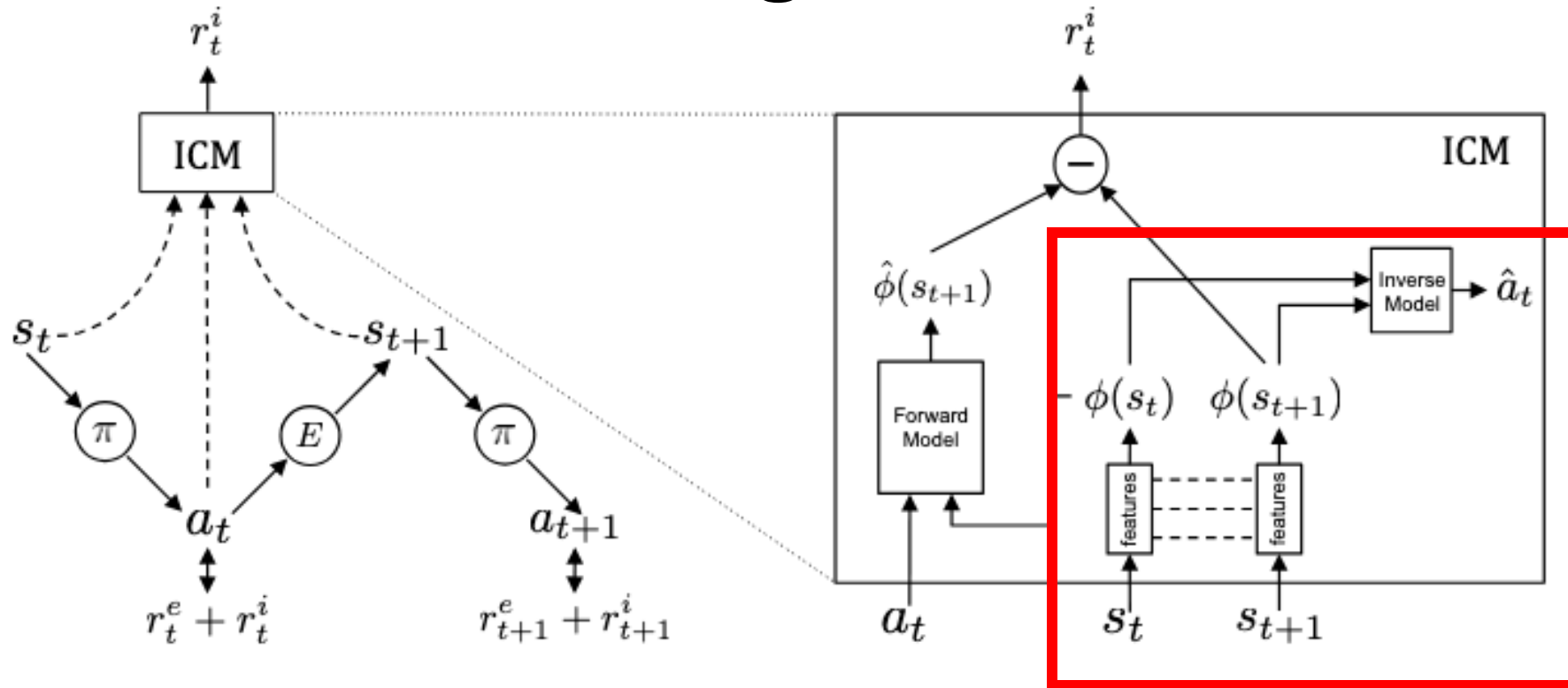
Intrinsic curiosity module



Inverse model used to train meaningful state representation (features)

- Goal: only model effect of **agent's own actions**!

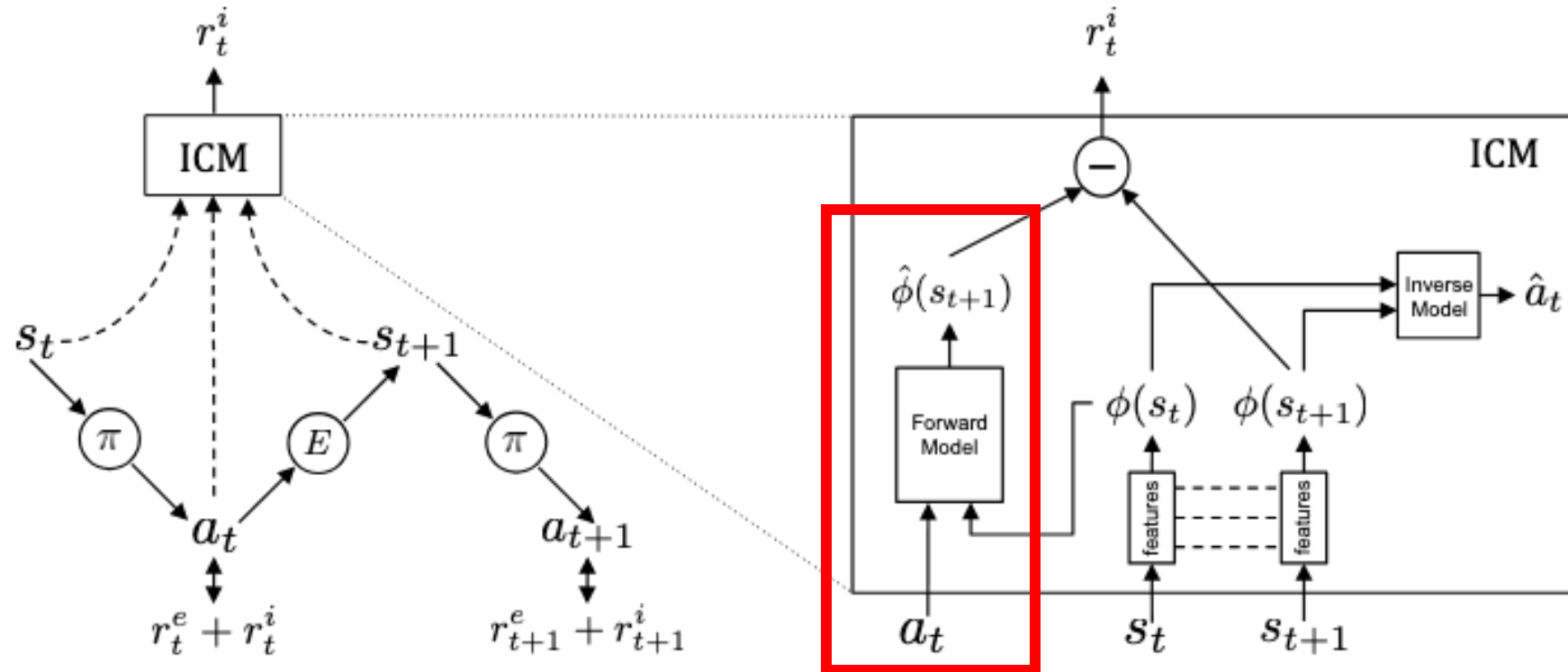
Only model effects of agent's actions



Predict what action was taken to go from $\phi(s_t)$ to $\phi(s_{t+1})$

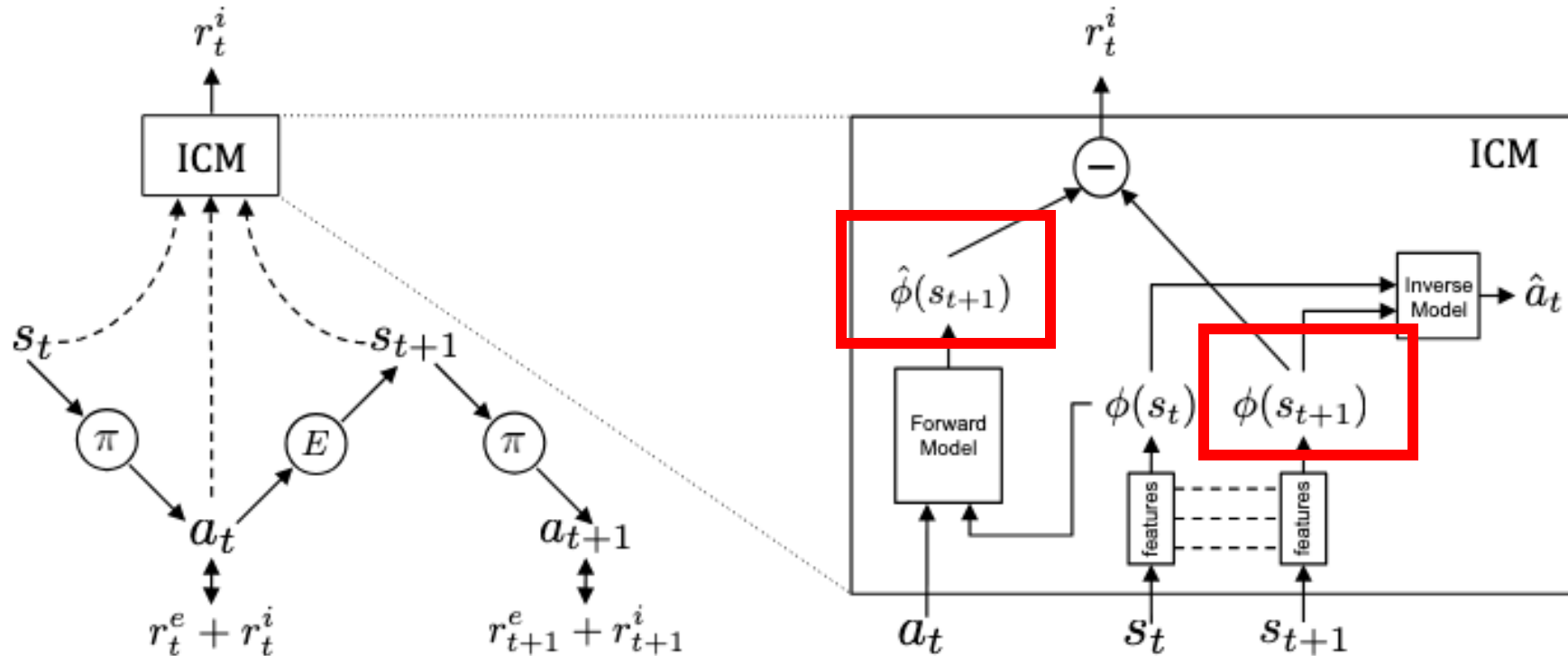
- $\phi(\cdot)$ is a feature encoder

Predict next state features



Predict $\phi(s_{t+1})$ given $\phi(s_t)$ and a_t

Use difference as a reward



Reward is obtained from $\hat{\phi}(s_{t+1}) - \phi(s_{t+1})$

Curiosity Driven Exploration by Self-Supervised Prediction

ICML 2017

Deepak Pathak, Pulkit Agrawal, Alexei Efros, Trevor Darrell
UC Berkeley

Using a teacher to guide exploration

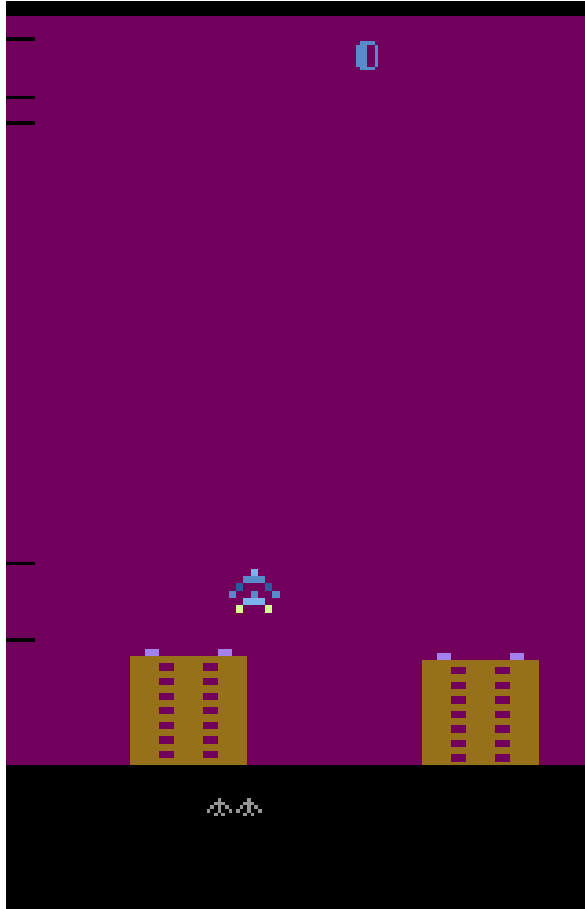
What if we have another agent that has learned a similar task?

- Task is *different* but shares some things in common

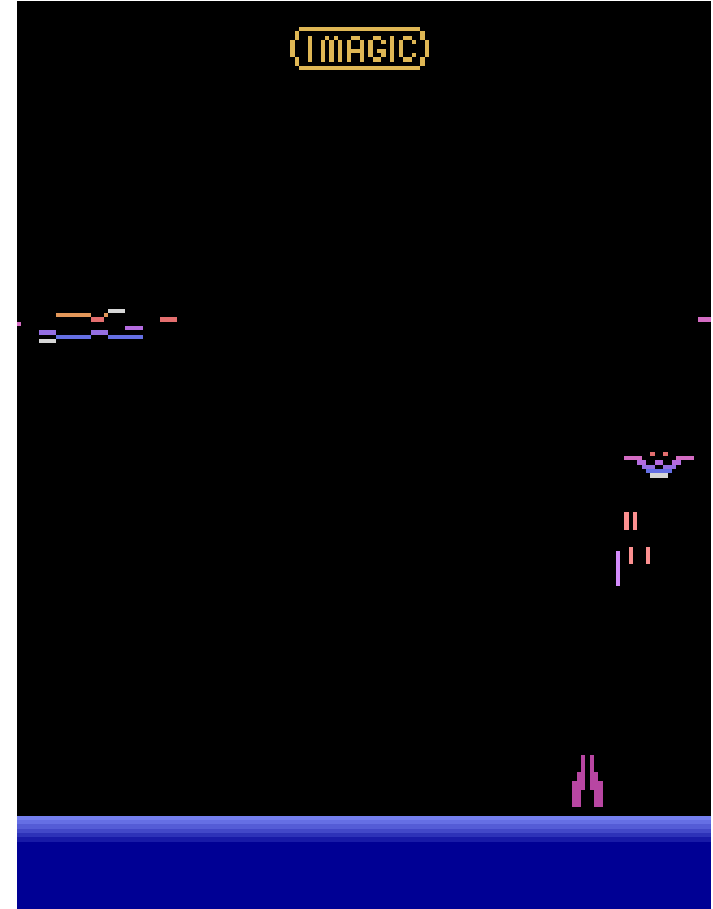
Can this agent (teacher) help our agent learn its own task?

Idea: teacher provides bonus reward to student when it enters states that are beneficial

Example in Atari



Air Raid
(Teacher)



Demon Attack
(Student)

Policy distillation

Distill the teacher's policy into the student's via bonus rewards

Use the teacher's critic to compute difference between current state and previous state

$$\text{Bonus} = V_{Teach}(s_{t+1}) - V_{Teach}(s_t)$$

Jump-start RL

Explore using the teacher's policy

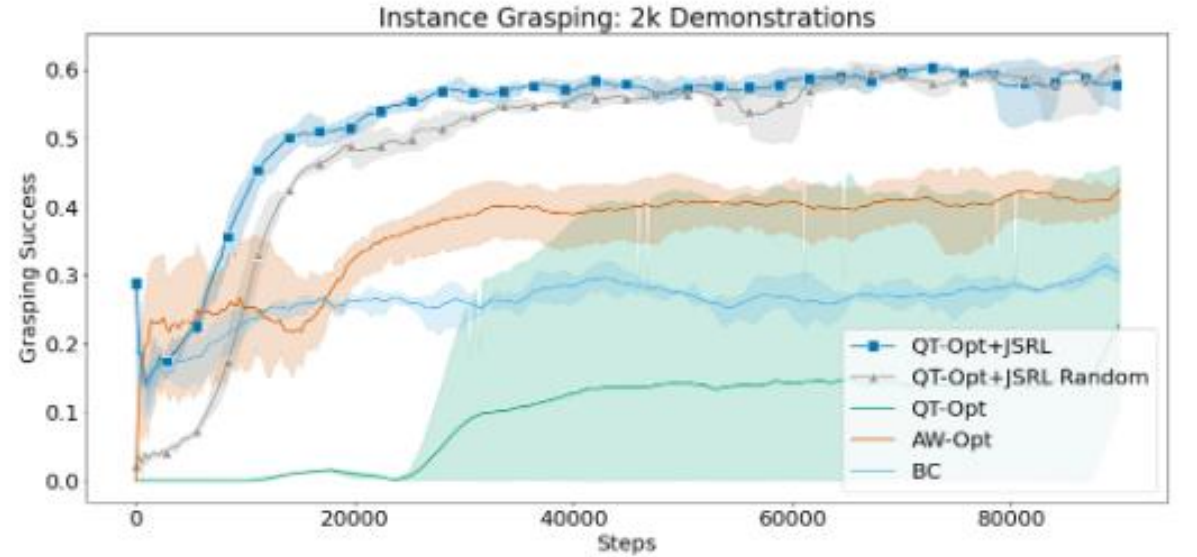
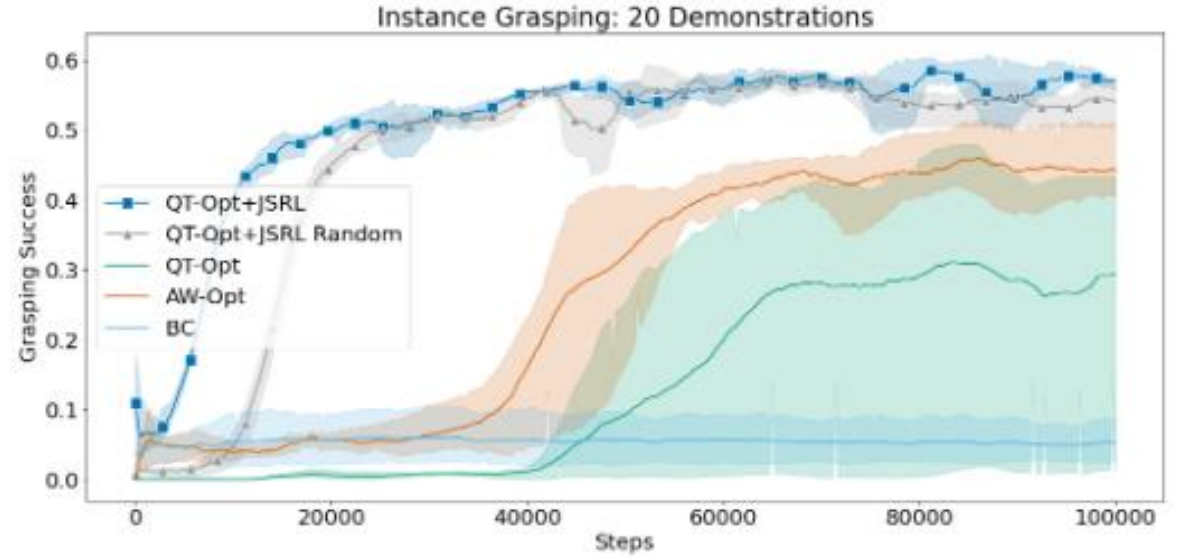
- Slowly transition from teacher to student as student improves
- Early learning = mostly teacher
- Late learning = mostly student

Unlike distillation, the teacher can be trained using **imitation learning**

- Only needs access to an offline dataset

Algorithm 1 Jump-Start Reinforcement Learning

- 1: **Input:** guide-policy π^g , performance threshold β , task horizon H , a sequence of initial guide-steps $H_1, H_2, \dots, H_n$, where $H_i \in [H]$ for all $i \leq n$.
 - 2: Initialize exploration-policy from scratch or with the guide-policy $\pi^e \leftarrow \pi^g$. Initialize Q -function $\hat{Q}$ and dataset $\mathcal{D} \leftarrow \emptyset$.
 - 3: **for** current guide step $h = H_1, H_2, \dots, H_n$ **do**
 - 4: Set the non-stationary policy $\pi_{1:h} = \pi^g, \pi_{h+1:H} = \pi^e$
 - 5: Roll out the policy π to get trajectory $\{(s_1, a_1, r_1), \dots, (s_H, a_H, r_H)\}$; Append the trajectory to the dataset $\mathcal{D}$.
 - 6: $\pi^e, \hat{Q} \leftarrow \text{TRAINPOLICY}(\pi^e, \hat{Q}, \mathcal{D})$
 - 7: **if** $\text{EVALUATEPOLICY}(\pi) \geq \beta$ **then**
 - 8: Continue
 - 9: **end if**
 - 10: **end for**
-



What are the problems with these methods?

What are the problems with these methods?

They assume that the student's task is sufficiently **in-distribution** for the teacher to act upon!

What happens if this isn't the case and the teacher gives bad advice?

- Can completely cripple the student's exploration
- Repeatedly lead the student to low reward regions
- May get stuck in local minima and unable to break out even after teacher stops giving advice

Introspective transfer learning

What if the teacher introspects over its own policy before giving advice?

- If advice doesn't help the student, don't give it!

Goal: avoid issuing bad advice

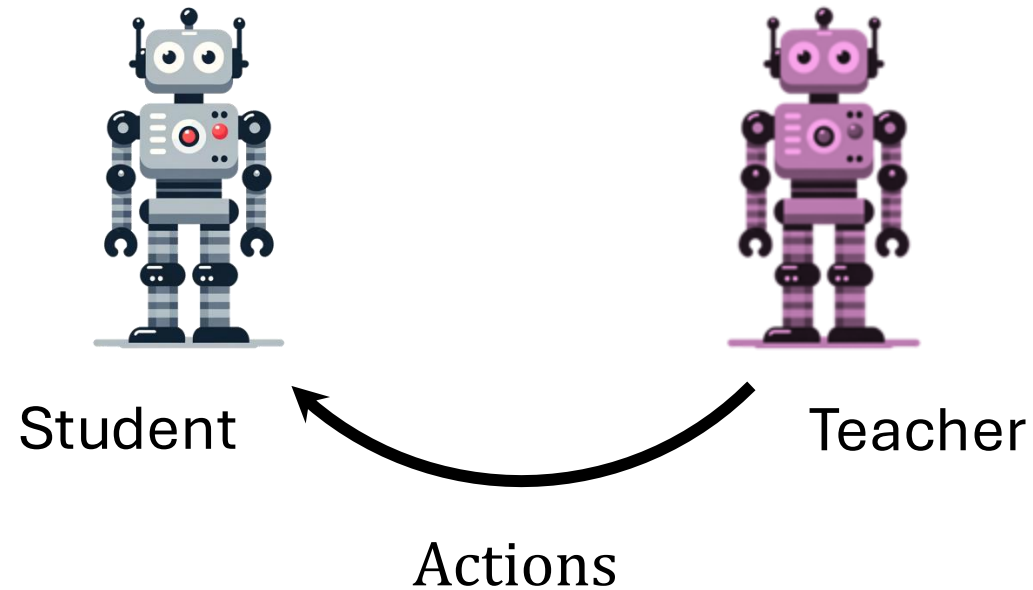
How do we know if advice helps?

- Compute expected reward in student's environment
- Out-of-distribution detection

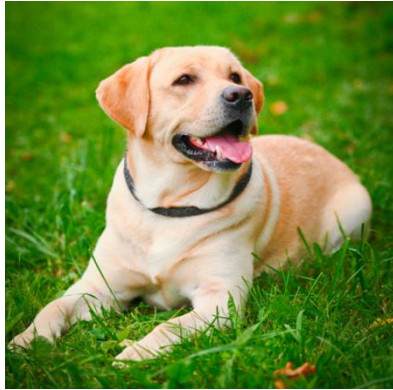
Inverse reinforcement learning

Recap: Imitation learning

The student tries to learn and **imitate** the teacher's actions



Supervised learning



Input x

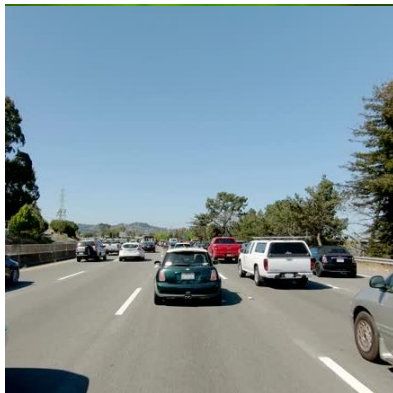


Model f_{θ}

"Labrador"

Output $y = f_{\theta}(x)$

Behavior cloning



Input x



Model f_{θ}

"Slow down"

Output $y = f_{\theta}(x)$

Inverse reinforcement learning

Given a set of demonstrations, instead of inferring policy (BC)
instead infer reward function

With reward function, perform standard reinforcement learning to
learn policy

This is another type of **imitation learning**

Inverse reinforcement learning assumptions

As in BC, assume access to a dataset $D = \{(s_t, a_t, s_{t+1})\}$

No reward is available!

Demonstrations are “good”

- Not necessarily optimal, but also not actively “bad”

Goal: learn teacher's reward function

Learn reward function for teacher policy π^* such that

$$\pi^* = \arg \max_{\pi \in \Pi} \mathbb{E}[r^*(s, a)]$$

Idea: given (state, action, next state) triplets, try to estimate what reward the teacher may be maximizing when choosing actions

- Literally *inverse* reinforcement learning
- Instead of learning policy from reward, learn reward from policy

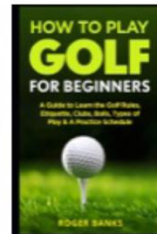
Why learn the reward function?

Since we have demonstrations, why not just use behavior cloning?

- Learning r^* (can be) easier than learning π^*
- Inspired by how humans learn

BC

Expert demonstrations



Supervised learning

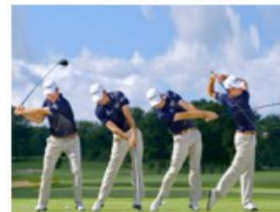
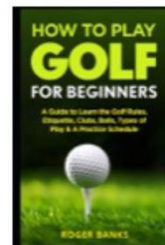


Estimated policy:

$\hat{\pi}$

IRL

Expert demonstrations



Reinforcement learning

Act in environment



Estimate the policy:

$\hat{\pi}$

Why learn the reward function?

The reward is the most **succinct**, **robust**, and **transferable** definition of a task – Ng and Russell, 2000

Remember the reward hypothesis?

“That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward).”

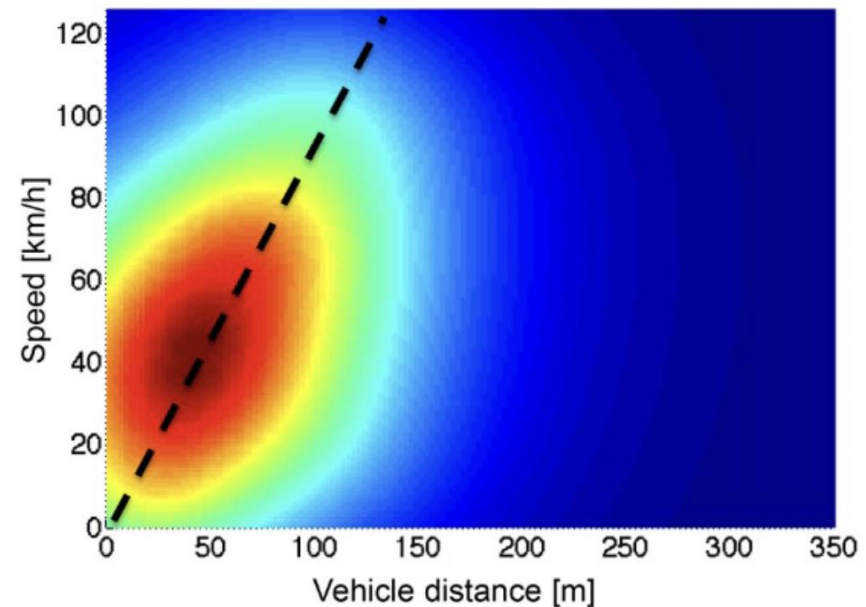
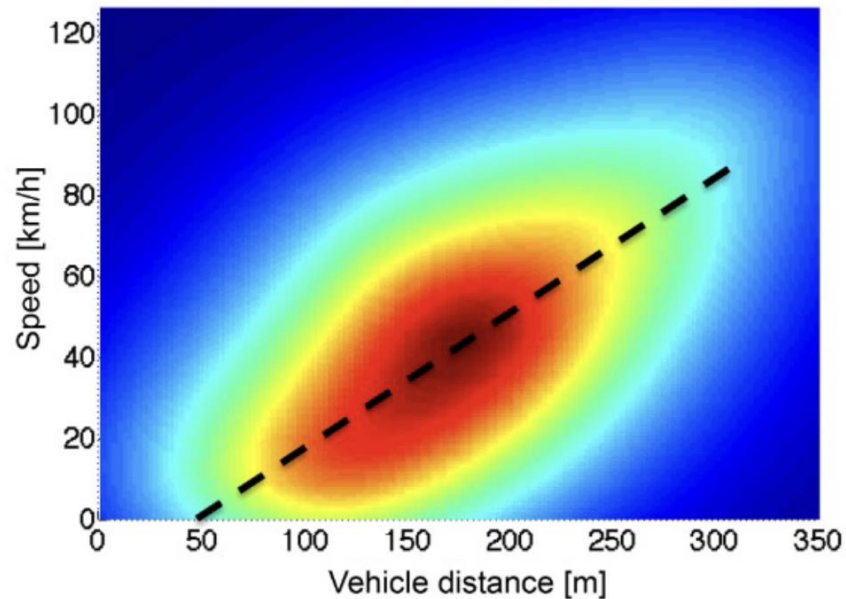
- Sutton and Barto, Chapter 3.2

Advantages:

- Interpretable (compared to policy)
- Can generalize or adapt to new environments

Disadvantages:

- Need extra steps to learn policy once reward is found



Reward functions from two different drivers in car-following problem

Linear reward formulation

Commonly used formulation, easy to optimize

$$R_{\theta}(s) = \theta_1 \phi_1(s) + \dots + \theta_d \phi_d(s) = \boldsymbol{\theta}^T \boldsymbol{\phi}(s)$$

θ = unknown weights, ϕ = known features

- Features could given as output by DNN

Linear reward formulation

Value function is linear w.r.t. “expected features” $\mu^\pi(s)$

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t r(s^{(t)}) \mid s^{(0)} = s \right] \\ &= \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{\theta} \cdot \boldsymbol{\phi}(s^{(t)}) \mid s^{(0)} = s \right] \\ &= \boldsymbol{\theta} \cdot \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{\phi}(s^{(t)}) \mid s^{(0)} = s \right] \\ &= \boldsymbol{\theta} \cdot \boldsymbol{\mu}^\pi(s) \end{aligned}$$

$$V^\pi(s) = \boldsymbol{\theta}^\top \boldsymbol{\mu}^\pi(s)$$

Linear reward formulation

The expected features can be estimated Monte Carlo-style

$$\begin{aligned}\boldsymbol{\mu}^\pi(s) &= \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{\phi}(s^{(t)}) \right] \\ &\approx \frac{1}{M} \sum_{m=1}^M \sum_{t=0}^{T_m} \gamma^t \boldsymbol{\phi}(s_m^{(t)})\end{aligned}$$

Solving IRL via linear programs

Problem: many reward functions which satisfy

$$\pi^* = \arg \max_{\pi \in \Pi} \mathbb{E}[r^*(s, a)]$$

How do we choose a reward function?

- Experts chosen action is better than all others

“Maximize the sum of differences between the quality of the optimal action and the quality of the next-best action”

$$\sum_{s \in \mathcal{S}} Q^{\pi}(s, a') - \max_{a \in \mathcal{A} \setminus a'} Q^{\pi}(s, a)$$



<https://youtu.be/M-QUkgk3HyE?si=JVgCXlu8QnuZ0RQo&t=4>

Maximum entropy IRL

Remember that IRL is ill-posed

- Many reward functions can correspond to the same policy
- Many mixtures of policies correspond to same feature expectation

Instead of arbitrary selecting a policy, e.g. max Q margin...

...use the **principle of maximum entropy!**

Principle of maximum entropy

The probability distribution which best represents the current state of knowledge about a system is the one with the largest entropy.

- E.T. Jaynes, 1957

Occam's Razor: *The simpler explanation is to be preferred.*

- The distribution with the largest entropy (uncertainty) makes the **least assumptions** about the true distribution

Example

Suppose in a robot navigation task the “expert” always drives near the northwest corner of the room.

Is it because...

- They prefer the northwest corner?
- They dislike the southeast corner?
- They just like moving in that direction?

MaxEnt IRL tries to resolve this ambiguity by assuming that the expert is **not deterministically optimal**, but instead **favors high reward paths** while behaving **stochastically**.

MaxEnt IRL with linear reward formulation

$$p(\tau|\theta) = \frac{1}{Z(\theta)} e^{\sum_{s_t \in \tau} \theta^T \phi(s_t)}$$

Normalization constant
(softmax dist. over trajectories)

Exponential sum of rewards = trajectories with
higher rewards are exponentially more likely

$$\theta^* = \arg \max_{\theta} L(\theta) = \arg \max_{\theta} \sum_{\tau_i \in \mathcal{D}} \log p(\tau|\theta)$$

MaxEnt IRL with linear reward formulation

Idea: make the state-visitation frequency under the learned policy match the state-visitation frequency of the expert.

Starting with randomly initialized reward function...

1. Compute optimal policy for current reward
2. Estimate expected state visitation frequency
3. Adjust reward function to match expert's state visitation frequency

Repeat until convergence

Generative adversarial imitation learning